

Helix OTA — Master Comprehensive Test-Coverage + Anti-Bluff Build-Out Plan

Revision: 1 Last modified: 2026-06-23T13:20:00Z Goal (operator mandate, 2026-06-23): genuine 100% real coverage by every supported test type (unit, integration, e2e, full-automation, stress, security, benchmark, chaos, ui/ux, perf/scaling) + Challenges (Challenges submodule) + HelixQA suites — all exercising **the real production-ready system running on the configured cluster**, every PASS backed by rock-solid captured physical evidence, comprehensive anti-bluff everywhere, no false positives. **Honesty (§11.4.6):** this is a multi-phase program, not a one-shot. Sources: the three audit docs in this directory (test-coverage-audit / cluster-and-system / challenges-helixqa-antibluff), each FACT-cited.

Current state (FACT, from the three audits)

- **Go layer is strong + genuinely anti-bluff** — measured `go test -cover` (health 100% ... `ota-validator/rollout` 100%), `-race/vet` clean, and **mutation-proven**: flipping the `OTA ed25519` signature check to always-accept KILLED 11 tests. No bluff tests in the sampled set.
- **The pgx/Postgres Repository is fully implemented** (real-DB is one env var away); store/rollout default coverage is understated only because the `-tags` integration suite wasn't run.
- **“Configured cluster” is aspirational for OTA** — one live podman host (thinker) runs the HelixTrack stack, NOT `ota-server`; no multi-node/k8s. Primary blocker: no combined `ota-server+postgres` compose stack for one-command real-system boot.
- **Challenges + HelixQA are powerful but not wired against the OTA server** (shell-dispatch bank only; the `Go cmd/helixqa` orchestrator + vision toolchain are uninstalled against OTA).
- **Anti-bluff pattern is correct but partial** — applied to ~2–3 of ~17 pre-build gates; `ab_pass_with_evidence` per-assertion helper unimplemented in `helix_ota`; 14 pre-build gates are bare anchor-presence greps.

Phase 0 — foundational enablers (highest leverage; unblock everything)

ID	Deliverable	Unblocks	Evidence on done
F-CLUSTER	<code>server/deploy/system.compose.yml</code> (<code>ota-server</code> + <code>postgres</code> , real envs + <code>/readyz</code>	<code>integration/e2e/full-auto/stress/</code>	the harness boots the stack, /

ID	Deliverable	Unblocks	Evidence on done
F- ANTIBLUFF-LIB	healthcheck) + a Go boot harness (mirror postgres_integration_test.go) via the containers brick pkg/ {boot, compose, health} → returns the live base URL	security/chaos against the REAL system (real HTTP + real DB, §11.4.27)	readyz→200, captured
	tests/lib/anti_bluff.sh — ab_pass_with_evidence <desc> <path> (refuses PASS unless path exists+non-empty), ab_skip_with_reason <closed-set> (§11.4.69)	mechanical per-assertion no-evidence-no-PASS across all tests/	self-test: PASS-without-evidence→FAIL, with-evidence→PASS
F-METAGATES	tests/meta/meta_test_<gate>.sh per functional gate (mutate→FAIL→restore→PASS) + tests/meta/run_all.sh into pre-build; implement CM-SEMGREP-WIRED (no fail-open)	every gate becomes bluff-proof (§1.1)	each meta-test: mutated→gate FAILs, restored→PASSes
F-CLUSTER-DEPLOY	a deploy/ota-system/ compose + distribute_stack.sh path that deploys ota-server+postgres to thinker = the real “cluster” target	“real system on the configured cluster” testing	podman ps (healthy) + / readyz→200 on thinker, captured

Phase 1 — per-test-type real coverage to 100% (risk-descending, §11.4.132)

Each closes with: real run against F-CLUSTER’s live system + captured evidence (§11.4.5/§11.4.69) + a paired §1.1 mutation routed through F-ANTIBLUFF-LIB.

1. **Integration (pgx/Postgres)** — run `-tags integration`, capture store/rollout real coverage → from UNCONFIRMED to measured. (*in progress*)
2. **e2e / full-automation** — tests/e2e/* against the live system stack (real HTTP+DB), aggregated into one mechanical runner; cover every flow (auth, device-register, artifact upload+signature-verify, rollout staged+halt, recall, telemetry).
3. **Security / DDoS** — extend `security_probes.sh` (authz/injection/JWT-tamper/the request-supplied-key-ignored trust boundary) + rate-limit saturation + go test -fuzz on upload/telemetry parsers.
4. **Stress / chaos (§11.4.85)** — HTTP load with p50/p95/p99 (wire tools/ loadtest); chaos: malformed/oversized artifact, replay, concurrent rollout, mid-write SIGKILL, postgres-down/network-fault recovery.

5. **Benchmark** — benchstat baseline registry for the 7 existing benchmarks + regression guard.
6. **Android (instrumentation/ui)** — Gradle/JaCoCo coverage for the two JVM bricks + on-device A/B (Cuttlefish cvd — already proven Tier-2, or RK3588) with captured evidence.
7. **cmd/* smoke** — boot each main.

Phase 2 — Challenges + HelixQA against the real system

- **AB-G4** new OTA challenges (each real dispatch + evidence artifact + paired mutation): bad-signature-rejected, request-key-ignored, telemetry-schema, rollout-staged+halt, chaos.
- **AB-G5** wire the Go cmd/helixqa orchestrator (native crash/ANR/step-validation/evidence) against the live OTA system; bridge UI recordings through HelixQA vision (helixqa-text/recvalidate/recording-analyzer) for read-the-screen PASS (§11.4.27/§11.4.160).
- Run a full HelixQA autonomous session over the helix_ota bank against F-CLUSTER's live system; capture the evidence ledger.

Phase 3 — anti-bluff everywhere (AB-G1..G3, woven through every phase)

Every test type + Challenge + HelixQA result MUST: cite a captured-evidence path (F-ANTIBLUFF-LIB refuses PASS otherwise); carry a paired §1.1 mutation (F-METAGATES); the §11.4.165 independent-verifier on the batch; the §11.4.163 media-validation pipeline for any recorded artifact. No gate without its mutation; no PASS without its evidence path.

Coverage ledger (live — updated as phases close)

Test type	Current real coverage	Target	Status
unit (Go)	server 47.9–100% per pkg; ota-* 98–100%	100% real	strong; store/rollout pending integration run
integration (pgx)	store 85.5% / rollout 83.1% MEASURED (real podman+Postgres on nezha, 2026-06-23)	100% of pgx paths	DONE — real PASS, evidence docs/qa/20260623-postgres-integration/
e2e / full-auto	challenge_operational 39/39 PASS against REAL Postgres (DB-proof: audit_logs 14 rows persisted,	all flows vs live + aggregated runner	black-box e2e DONE vs real DB ; signed-pipeline-vs-

Test type	Current real coverage	Target	Status
	device_groups=0 end-of-run-delete real; docs/qa/20260623-e2e-live-system/). 4/5 suites self-hosting-by-design (must own the artifact pubkey to sign) → need a caller-pubkey F-CLUSTER mode for signed-pipeline-against-live.		live = new sub-item
anti-bluff per-assertion helper	none	ab_pass_with_evidence everywhere	F-ANTIBLUFF-LIB DONE (8/8 guard, mutation-proven)
security/ddos	59 hard asserts + rate-limit + runtime trust-boundary 4/4 vs live (request-supplied-key IGNORED) + Go fuzz DONE (4 targets, 10.8M execs, 0 crashers , property-based: round-trip/antisymmetry/hex-oracle; docs/qa/20260623-fuzz/)	+ saturation	trust-boundary + fuzz DONE ; saturation pending
stress/chaos	HTTP load + CHAOS vs live DONE . Load: p99 14.32ms @ 5,540 req/s, zero 5xx. Chaos 4/4 survive+recover (docs/qa/20260623-chaos-live/); postgres-kill→reconnect (no corruption, validates the main.go retry fix), malformed/oversized→400, 12-race idempotent register→1 winner+exactly-one-device, 400-conn churn→recovers	full	DONE
benchmark	benchstat baseline registry + negation-proven guard DONE (7 benches,	tighter ns gate (dedicated runner)	DONE

Test type	Current real coverage	Target	Status
Android instrumentation/ui	e.g. MemoryFindDelta 196ns/0-allocs; threshold calibrated on measured variance — allocs >25%, ns >2×; docs/benchmarks/ + docs/qa/20260623-benchmarks/) JaCoCo MEASURED (wired both bricks): ota-update-engine-bridge LINE 100% (113/113, 27 tests), ota-android-agent LINE 91.18% (362/397, 47 tests; json-codec branch 55.6% → now 100%/ 100% via 54 new tests, docs/qa/20260623-json-coverage/); docs/qa/20260623-android-jacoco/	on-device A/B	JVM coverage DONE (json 100%); on-device pending (needs device)
Challenges	15 HOTA banks (shell-dispatch)	+ OTA holes + paired mutations	partial
HelixQA	bank-runner gates only	Go orchestrator + vision against live	gap
anti-bluff gates	+4 now bluff-proof via paired §1.1 meta-tests (coverage-minimum / semgrep-wired / 2 regression guards / evidence-lib) — tests/meta/framework wired into pre-build; semgrep fail-open FIXED (no silent pass; semgrep on PATH → gate passes); a real intermittent flake found+fixed (§11.4.50, 10/10 det). docs/qa/20260623-metagates/	all ~17 bluff-proof	F-METAGATES + propagation-gate batch DONE (meta suite 5/5; ALL 14 propagation gates now §1.1-paired, data-driven from pre_build source so new gates auto-covered; no other unpaired grep-gate exists); meta suite + benchmark guard

Execution discipline

Subagent-driven (§11.4.70), parallel background streams (§11.4.103, paced for the API throttle), each deliverable producing rock-solid captured evidence under docs/qa/<run-id>/ (§11.4.83), honest §11.4.6 (measured numbers as FACT, unrun as UNCONFIRMED, blocked-with-reason never faked). Phase 0 first (it unblocks everything); then Phase 1 risk-descending; Phases 2–3 woven in.